

Unity URP 렌더 최적화 이론 강의 노트



실습 전에 이해해야 할 렌더링 비용, 프로파일링, URP 설정

이 문서의 기준

프로젝트 기준: Unity 6000.3.9f1, Universal Render Pipeline 17.3.0

목표: 개발자 지원 포트폴리오용 렌더 최적화 테스트 필드에 들어가기 전 이론 정리

주의: 특정 회사의 내부 모델 포맷이나 파이프라인은 공개 자료만으로 단정하지 않음

출처: 공식 문서 중심, 모든 외부 문서는 2026-07-20 기준 확인

0강. 사실과 가정을 분리한다

강의 시작 전에 먼저 지켜야 할 원칙

- 프로젝트 사실: 현재 프로젝트는 Unity 6000.3.9f1, URP 17.3.0 기준이다.
- 프로젝트 사실: 기본 씬은 카메라, 방향성 조명, 볼륨 정도만 있는 초기 상태에 가깝다.
- 직무 해석: 공고 이미지에는 Unity, 리소스 최적화, 부하/기능 테스트, 스튜디오 운영 유지보수가 보인다.
- 주의: 특정 기업이 VRM, FBX, 자체 포맷 중 무엇을 쓰는지 공개 자료만으로 단정하지 않는다.
- 따라서 포트폴리오는 특정 포맷이 아니라 Unity 3D 캐릭터/라이브 씬 최적화로 설계한다.

오늘의 결론

이론 수업의 목표는 '어떤 옵션을 고면 빨라진다'를 외우는 것이 아니라, 병목을 측정하고 원인을 설명하는 언어를 갖추는 것이다.

1강. 렌더 최적화는 프레임 예산 관리다

- 게임 화면은 매 프레임 반복해서 계산되고 그려진다.
- 60 FPS 목표라면 한 프레임에 사용할 수 있는 총 시간은 약 16.67ms다.
- CPU, GPU, 스크립트, 애니메이션, 렌더링, 메모리 할당이 모두 이 예산을 나눠 쓴다.
- 최적화의 첫 질문은 '느린가?'가 아니라 '어느 구간이 예산을 초과하는가?'다.

60 FPS



30 FPS



24 FPS



강의 메모

FPS는 결과값이고, Frame Time은 원인 분석에 더 가까운 숫자다. 포트폴리오에서는 FPS와 ms를 함께 기록한다.

2강. 화면 한 장이 그려지는 큰 흐름

실시간 3D 렌더링은 대체로 CPU가 장면 상태를 정리하고, 그래픽 API 명령을 만들고, GPU가 정점/래스터화/픽셀 단계에서 이미지를 만들어내는 구조로 이해할 수 있다.



- Vertex 단계: 메시의 정점이 변환되고, 스키닝이나 정점 기반 계산이 들어갈 수 있다.
- Rasterization: 삼각형이 화면의 fragment/pixel 후보로 쪼개진다.
- Fragment/Pixel 단계: 텍스처 샘플링, 조명, 그림자, 투명도, 후처리 비용이 커지기 쉽다.
- Depth/Blend 단계: 깊이 테스트, 스텐실, 블렌딩 같은 per-sample 작업이 일어난다.

3강. CPU 병목과 GPU 병목을 구분한다

CPU 쪽 렌더 병목

Draw Call 준비, SetPass 전환, 스크립트, 애니메이션, 오브젝트 탐색, 컬링 준비 등이 누적될 때 나타난다.

S5/S6

GPU 쪽 렌더 병목

해상도, fragment shader, texture bandwidth, render target, shadow map, post process, overdraw 비용이 커질 때 나타난다.

S11/S12

증상	우선 의심	첫 확인
해상도 낮추면 빨라짐	GPU fill/fragment	Render Scale, Post, Overdraw
Draw/SetPass가 큼	CPU render setup	Rendering Profiler, Frame Debugger
그림자 켜면 급락	Shadow pass	Shadow caster, light count
캐릭터 수에 비례	Skinning/material	SkinnedMeshRenderer, material count

4강. Draw Call, SetPass, Batch

- Draw Call은 Unity가 화면에 렌더링하라고 그래픽스 쪽에 보내는 그리기 명령이다.
- SetPass는 렌더링에 사용할 shader pass가 바뀐 횟수로, 상태 전환 비용을 이해하는 데 중요하다.
- Batch는 Unity가 여러 렌더 작업을 묶어 처리한 단위다.
- Rendering Profiler는 Batches, SetPass Calls, Triangles, Vertices, Draw Calls 같은 지표를 보여준다.

포트폴리오 문장 예시

서로 다른 머티리얼을 많이 사용한 구역에서 SetPass Calls가 증가했고, 동일 shader variant와 머티리얼 공유를 적용한 뒤 CPU 렌더 준비 비용을 비교했다.

5강. 묶어서 그리는 전략들

Static Batching

움직이지 않는 오브젝트를 큰 메시처럼 묶어 CPU 렌더 비용을 줄이는 방향. 메모리 비용과 정적 조건을 고려한다.

S6

SRP Batcher

SRP에서 같은 shader variant를 쓰는 머티리얼의 CPU 렌더 준비 비용을 줄이는 경로. URP/HDRP에서 사용 가능하다.

S9

GPU Instancing

같은 mesh와 material의 복사본을 한 draw call로 렌더링하는 최적화. SRP Batcher와 우선순위/호환 조건을 확인해야 한다.

S10

- 처음 실습에서는 같은 큐브 100/500/1000개, 같은 머티리얼/다른 머티리얼 조건을 나눠 테스트한다.
- 캐릭터는 SkinnedMeshRenderer가 많아질 수 있으므로, 단순 인스턴싱만으로 모든 문제가 해결된다고 보면 안 된다.
- 좋은 포트폴리오는 '어떤 기법을 썼다'보다 '이 조건에서는 이 기법이 더 맞았다'를 보여준다.

6강. URP Asset은 렌더링 품질의 조종판이다

Rendering

Depth Texture, Opaque Texture, Store Actions

S3

Quality

HDR, MSAA, Render Scale, Upscaling

S3

Lighting

Main Light, Additional Lights

S3

Shadows

Distance, Resolution, Cascade, Soft Shadow

S3

Post

Bloom, SSAO, Tonemapping 등

S3

현재 프로젝트 힌트

이미 PC_RPAsset과 Mobile_RPAsset이 있으므로, 이론 이후 PC/Mobile 비교 실험으로 바로 이어갈 수 있다.

S2/S3

7강. 가장 먼저 실험할 URP 레버

Render Scale

렌더 타겟 해상도를 조절한다. 낮추면 GPU 비용이 줄 수 있지만 선명도가 희생된다.

HDR

넓은 밝기 범위와 Bloom 표현에 유리하지만 낮은 사양에서는 계산을 줄이기 위해 끌 수 있다.

MSAA

기하 경계의 계단 현상을 줄이지만 샘플 수가 늘수록 비용이 증가한다.

Depth/Opaque Texture

효과 구현에 필요할 수 있지만 추가 텍스처 생성/복사 비용을 만든다.

Store Actions

특히 모바일/tile-based GPU에서 메모리 대역폭 비용과 연결될 수 있다.

8강. Forward, Forward+, Deferred

Forward

기본 경로. 조명이 많지 않거나
모바일/저사양 플랫폼에서 우선 고려할
수 있다.

S4

Forward+

많은 조명이 필요하지만 Deferred가
느린 플랫폼 등에서 검토한다.

S4

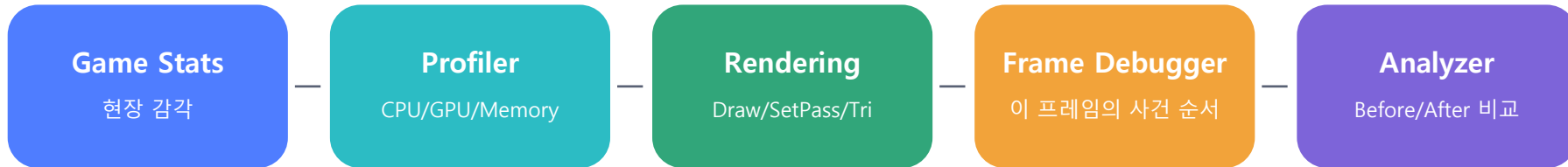
Deferred

조명 수 처리에 강점이 있지만 G-buffer
등 추가 렌더 타킷 비용을 고려해야
한다.

S4

- URP 문서는 프로젝트 유형과 목표 하드웨어에 따라 렌더링 경로를 고르라고 설명한다.
- Forward 계열은 MSAA 지원 측면에서 유리한 경우가 있고, Deferred는 모바일에서 추가 pass 비용이 커질 수 있다.
- 처음 포트폴리오는 경로 전환보다 URP Asset 옵션과 조명/그림자 비용을 먼저 측정하는 편이 안전하다.

9강. 프로파일링 도구 사다리



- Profiler: CPU, GPU, Rendering, Memory 등 모듈별로 프레임 데이터를 확인한다.
- Rendering Profiler: Batches, SetPass, Draw Calls, Triangles, Vertices, RenderTexture, Shadow Casters 같은 렌더 지표를 확인한다.
- Frame Debugger: 한 프레임을 멈추고 어떤 렌더 이벤트가 어떤 순서로 실행되는지 확인한다.
- Profile Analyzer: 여러 프레임 데이터를 집계하고 두 데이터 세트를 나란히 비교한다.
- Memory Profiler: 스냅샷을 캡처해서 native/managed 메모리, 할당, 누수 의심 지점을 본다.

10강. 측정 규칙이 없으면 최적화도 없다

- 같은 씬, 같은 카메라 경로, 같은 해상도, 같은 품질 설정에서 비교한다.
- 처음 몇 초는 워밍업으로 버리고, 일정 시간의 평균/중앙값/최저 구간을 기록한다.
- Editor Play Mode 수치는 참고용이다. 가능하면 목표 플랫폼 빌드에서도 측정한다.
- VSync, 해상도, 품질 레벨, Development Build 여부를 로그에 남긴다.
- 자동화 단계에서는 Performance Testing Extension으로 반복 측정 기반을 만들 수 있다.

기록 템플릿

Case: Shadow Distance
Scene: Benchmark_Field
Quality: Mobile_RPAsset
Resolution: 1920x1080
Duration: 60s
Avg FPS:
CPU Main ms:
GPU ms:
Draw Calls:
SetPass:
Note:

11강. 메시 비용: 삼각형, 정점, LOD, 컬링

Triangles / Vertices

Rendering Profiler에서 프레임 동안 처리된 삼각형과 정점 수를 확인한다.

S6

LOD

먼 오브젝트에 낮은 디테일 메시를 사용해 GPU 작업을 줄이는 기법이다.

S14

Occlusion Culling

다른 오브젝트에 완전히 가려진 Renderer의 불필요한 렌더 계산을 줄인다.

S13

- 벤치마크 필드에는 '보이는 오브젝트 수'와 '가려지는 오브젝트 수'를 분리한 구역이 필요하다.
- LOD는 멀리 있는 메시의 삼각형 수를 낮추는 전략이므로 고정 카메라 거리 조건이 중요하다.
- Occlusion Culling은 CPU 런타임 계산과 메모리 데이터가 필요하므로, 실내/복도형 구조처럼 가림이 명확한 씬에서 효과를 확인한다.

12강. 머티리얼과 셰이더 비용

- Mesh Renderer와 Skinned Mesh Renderer는 Materials 목록을 가진다. 머티리얼 수와 sub-mesh 구조는 렌더 이벤트 수에 영향을 줄 수 있다.
- SRP Batcher는 같은 shader variant를 쓰는 머티리얼들의 CPU 렌더 준비 비용을 낮추는 데 도움이 된다.
- 커스텀 셰이더는 텍스처 대역폭, 버퍼 대역폭, ALU 연산, fragment shader 반복 계산을 확인해야 한다.
- fragment shader에서 매 픽셀 계산하는 값을 vertex shader나 C#으로 옮길 수 있는지 검토한다.

처음 만들 테스트

동일 mesh 300개를 같은 Lit material로 배치한 구역과, 서로 다른 material 300개로 배치한 구역을 나누고 Rendering Profiler / Frame Debugger로 SetPass와 draw event 차이를 확인한다.

13강. 투명 렌더링과 Overdraw

Opaque

깊이 테스트와 렌더 순서 최적화가 비교적 잘 맞는다. 불투명 오브젝트는 먼저 정리하기 좋다.

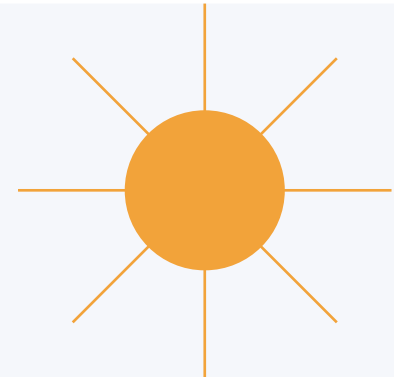
Transparent

블렌딩과 정렬이 필요하고, 가려진 픽셀도 여러 번 계산될 수 있어 overdraw 비용을 만든다.

- 캐릭터 머리카락, 장식, 이펙트, 유리/물 같은 표현은 투명 렌더링 비용을 만들 수 있다.
- Opaque Texture는 투명 셰이더의 굴절/열기/유리 효과에 사용할 수 있지만, 장면 스냅샷 생성 비용을 고려해야 한다.
- 투명 구역 실습에서는 카메라에 겹쳐 보이는 알파 오브젝트 수를 늘려 Frame Debugger와 GPU 시간을 비교한다.

14강. 조명과 그림자는 가장 비싼 품질 옵션 중 하나다

- 그림자 렌더링 시간은 shadow caster 수, shadow receiver 수, shadow-casting light 수, cascade count, shadow resolution, soft shadow 설정의 영향을 받는다.
- Point Light 그림자는 여섯 방향의 shadow map이 필요하므로 모바일에서는 특히 비용을 조심해야 한다.
- 멀리 있는 조명의 그림자를 끄거나, 카메라 거리 조건으로 실시간 조명을 비활성화하는 전략을 검토할 수 있다.
- 정적 오브젝트는 baked shadow / Shadowmask / Light Probe 조합을 비교할 수 있다.



Light/Shadow 실험

Shadow Distance, Cascade Count, Resolution, Soft Shadow, Additional Light Shadow를 하나씩 바꿔 비교한다.

15강. 텍스처와 메모리는 성능의 바닥 체력이다

Texture Bandwidth

셰이더가 텍스처를 많이 읽고 쓰면 GPU 대역폭 한계에 가까워질 수 있다.

S11

Mipmaps

Unity는 텍스처 읽기 최적화 전략 중 하나로 mipmap 사용을 제시한다.

S11

Memory Snapshot

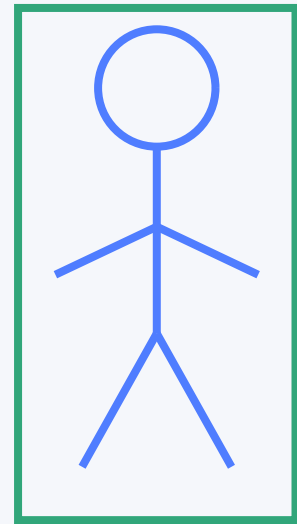
Memory Profiler는 메모리 스냅샷을 캡처하고 비교해 native/managed 할당을 본다.

S17

- Rendering Profiler는 사용 텍스처와 RenderTexture 관련 메모리 지표도 제공한다.
- Render Scale, HDR Precision, MSAA, Opaque/Depth Texture는 렌더 타겟 크기와 대역폭 비용에 연결된다.
- 캐릭터 포트폴리오에서는 텍스처 해상도, 압축, 머티리얼 슬롯 수, normal/emission 사용 여부를 함께 기록한다.

16강. 캐릭터 렌더링은 정적 메시와 다르다

- Skinned Mesh Renderer는 뼈대와 bind pose를 가진 deformable mesh, blend shape, cloth simulation 등을 렌더링한다.
- Bone influence 수가 많을수록 움직임 품질은 좋아질 수 있지만 계산 자원이 더 필요할 수 있다.
- Update When Offscreen은 보이지 않는 동안에도 bounds를 매 프레임 계산하게 하므로, 기본적으로 성능 이유로 꺼져 있다.
- BlendShapes는 표정/입 모양 같은 값에 쓰이며, 캐릭터 수가 늘 때 비용을 따로 관찰해야 한다.
- 캐릭터 테스트는 '모델 포맷'이 아니라 SkinnedMeshRenderer, material count, blend shape, shadow caster 수를 중심으로 설계한다.



Bounds / Bones / BlendShapes

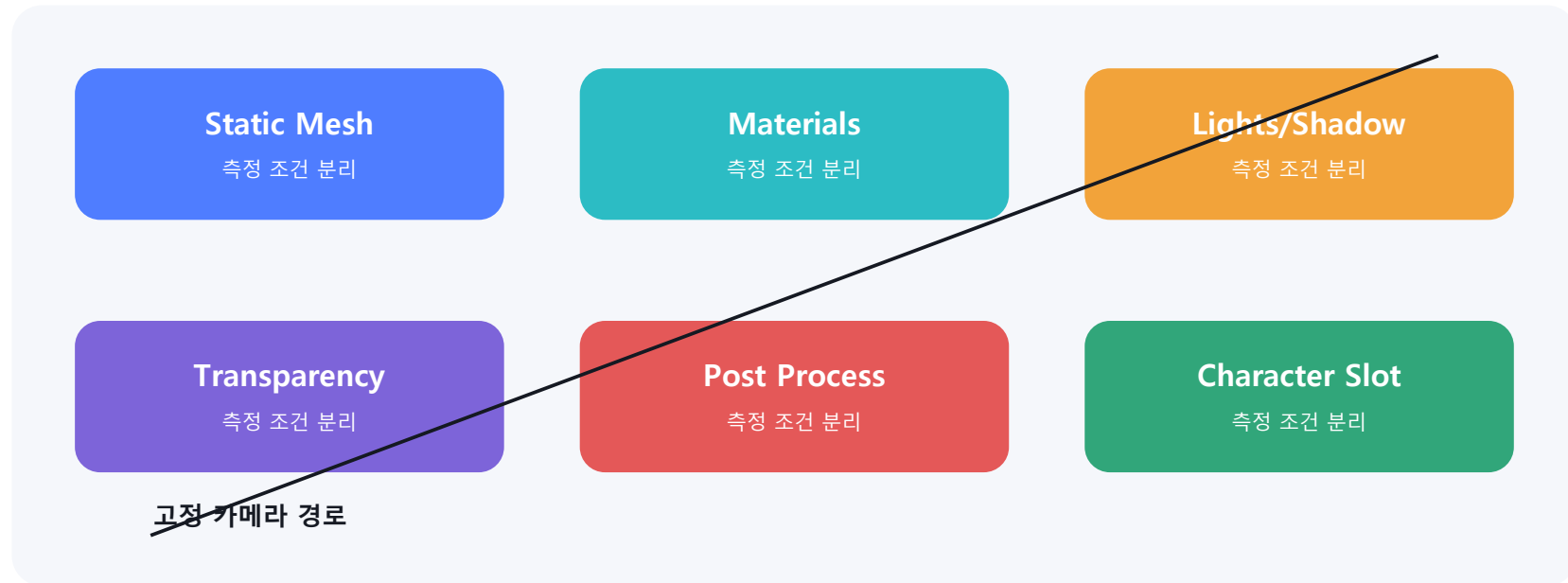
17강. 특정 포맷을 단정하지 않는 설계

- VTuber/아바타 분야에는 VRM, FBX, glTF, 자체 변환 포맷 등 여러 가능성이 있다.
- 공개 공개 이미지만으로 특정 기업이 어떤 포맷을 쓴다고 단정할 수 없다.
- 지원 포트폴리오는 포맷 이름보다 Unity에서 캐릭터 렌더링 비용을 분석하고 개선하는 능력을 보여줘야 한다.
- 그래서 테스트 필드는 'Character Slot'을 비워두고, 어떤 캐릭터 에셋도 나중에 넣을 수 있게 만든다.

지원서에 쓸 표현

Unity 기반 3D 캐릭터 라이브 씬을 대상으로, 캐릭터 렌더러/머티리얼/조명/URP 품질 설정의 비용을 측정하고 PC/Mobile 프로파일에 맞춰 최적화하는 테스트 환경을 구축했습니다.

18강. 이론을 실험장으로 바꾸면 Benchmark Field가 된다



- Static Mesh 구역: 오브젝트 수와 배칭 전략을 테스트한다.
- Materials 구역: 같은 shader/material과 서로 다른 material 조건을 비교한다.
- Lights/Shadow 구역: shadow distance, cascade, resolution, additional light shadow를 비교한다.
- Transparency/Post 구역: overdraw, Opaque Texture, Bloom/SSAO 등 화면 공간 비용을 확인한다.
- Character Slot: 포맷과 무관하게 SkinnedMeshRenderer 기반 캐릭터 비용을 나중에 측정한다.

19강. 첫 실습 전 로드맵

- 1** **폴더/문서** Assets/_Project와 Docs 구조를 만든다.
- 2** **Benchmark_Field** Primitive만으로 테스트 구역을 만든다.
- 3** **HUD** FPS, frame time, quality, render scale을 표시한다.
- 4** **수동 측정** Profiler와 Frame Debugger로 첫 baseline을 기록한다.
- 5** **비교** PC_RPAsset과 Mobile_RPAsset 차이를 문서화한다.
- 6** **확장** Profile Analyzer, Memory Profiler, 캐릭터 에셋으로 확장한다.

20강. 최소 용어 사전

Frame Time

한 프레임을 완성하는 데 걸린 시간. 60 FPS 기준 약 16.67ms 이하가 목표.

Draw Call

Unity가 화면에 그리라고 보내는 렌더 명령.

SetPass

렌더링에 사용할 shader pass가 바뀐 횟수.

Batch

Unity가 렌더 작업을 묶어 처리한 단위.

SRP Batcher

SRP에서 같은 shader variant 렌더 준비 비용을 줄이는 경로.

Render Scale

게임 렌더 타깃 해상도를 조절하는 URP 품질 설정.

Shadow Cascade

원거리/근거리 그림자 품질을 나눠 관리하는 그림자 방식.

Skinned Mesh

뼈대/블렌드셰이프 등으로 변형되는 캐릭터 메시.

21강. 실습 전에 외워야 할 질문

- ☐ 지금 병목은 CPU인가 GPU인가?
- ☐ 측정은 Editor인가 Build인가, 목표 하드웨어인가?
- ☐ FPS 말고 frame time ms를 기록했는가?
- ☐ Draw Calls, SetPass, Batches, Triangles, Vertices 중 무엇이 변했는가?
- ☐ URP 옵션 하나만 바꾸고 비교했는가?
- ☐ 품질 손실과 성능 개선을 둘 다 설명할 수 있는가?
- ☐ 캐릭터 비용을 모델 포맷이 아니라 SkinnedMeshRenderer/Material/BlendShape/Bone 기준으로 봤는가?

다음 수업 예고

이론을 끝낸 뒤 첫 실습은 Benchmark_Field 씬과 성능 HUD를 만드는 것이다.

참고문헌 및 이미지 출처

확인일: 2026-07-20. 본 PDF의 도식은 외부 이미지를 복제하지 않고 강의용으로 직접 제작한 벡터 도식이며, 도식 아래의 근거 태그는 아래 문헌을 의미한다.

- [S1] Project Unity version**
local project file - ProjectSettings/ProjectVersion.txt
- [S2] Project package manifest**
local project file - Packages/manifest.json
- [S3] Universal Render Pipeline Asset | Universal RP 17.0**
Unity Technologies - <https://docs.unity.cn/Packages/com.unity.render-pipelines.universal%4017.0/manual/universallrp-asset.html>
- [S4] Choose a rendering path in URP | Unity 6.0**
Unity Technologies - <https://docs.unity3d.com/6000.0/Documentation/Manual/urp/rendering-paths-comparison.html>
- [S5] Profiler window reference | Unity 6.0**
Unity Technologies - <https://docs.unity3d.com/6000.0/Documentation/Manual/ProfilerWindow.html>
- [S6] Rendering Profiler module reference | Unity 6.0**
Unity Technologies - <https://docs.unity3d.com/6000.0/Documentation/Manual/ProfilerRendering.html>
- [S7] Debug a frame | Unity 6.0 Frame Debugger**
Unity Technologies - <https://docs.unity.cn/6000.0/Documentation/Manual/FrameDebugger-debug.html>
- [S8] Profiling tools reference | Unity 6.0**
Unity Technologies - <https://docs.unity3d.com/6000.0/Documentation/Manual/performance-profiling-tools.html>
- [S9] Scriptable Render Pipeline Batcher**
Unity Technologies - <https://docs.unity.cn/Manual/SRPBatcher.html>
- [S10] Introduction to GPU instancing | Unity 6.0**
Unity Technologies - <https://docs.unity3d.com/6000.0/Documentation/Manual/GPUInstancing.html>
- [S11] Optimize custom shaders | Unity 6.0**
Unity Technologies - <https://docs.unity3d.com/6000.0/Documentation/Manual/SL-ShaderPerformance.html>

참고문헌 계속

확인일: 2026-07-20. 본 PDF의 도식은 외부 이미지를 복제하지 않고 강의용으로 직접 제작한 벡터 도식이며, 도식 아래의 근거 태그는 아래 문헌을 의미한다.

[S12] Optimize shadow rendering in URP | Unity 6.0

Unity Technologies - <https://docs.unity3d.com/6000.0/Documentation/Manual/shadows-optimization.html>

[S13] Occlusion culling | Unity 6.0

Unity Technologies - <https://docs.unity3d.com/6000.0/Documentation/Manual/OcclusionCulling.html>

[S14] Level of Detail for meshes | Unity 6.0

Unity Technologies - <https://docs.unity.cn/6000.0/Documentation/Manual/LevelOfDetail.html>

[S15] Mesh Renderer component reference | Unity 6.0

Unity Technologies - <https://docs.unity3d.com/6000.0/Documentation/Manual/class-MeshRenderer.html>

[S16] Skinned Mesh Renderer component reference | Unity 6.0

Unity Technologies - <https://docs.unity3d.com/6000.0/Documentation/Manual/class-SkinnedMeshRenderer.html>

[S17] Memory Profiler package 1.0

Unity Technologies - <https://docs.unity.cn/Packages/com.unity.memoryprofiler%401.0/manual/index.html>

[S18] Profile Analyzer package 1.2

Unity Technologies - <https://docs.unity.cn/Packages/com.unity.performance.profile-analyzer%401.2/manual/index.html>

[S19] Performance Testing Extension for Unity Test Framework 3.0

Unity Technologies - <https://docs.unity.cn/Packages/com.unity.test-framework.performance%403.0/manual/index.html>

[S20] Pipeline Stages (Direct3D 10)

Microsoft Learn - <https://learn.microsoft.com/en-us/windows/win32/direct3d10/d3d10-graphics-programming-guide-pipeline-stages>

[S21] Rendering Pipeline Overview

Khronos OpenGL Wiki - https://wikis.khronos.org/opengl/Rendering_Pipeline_Overview